MeshSentry: Lightweight Collaborative Intrusion Detection for ESP-NOW Mesh Networks on the ESP32-S3

Nabil Litayem
[TODO: Department], [TODO: University]
[TODO: City, Country]
nabil.litayem@gmail.com

***Abstract*—ESP-NOW lets low-cost ESP32 devices build connectionless multi-hop sensor networks, but these networks have no intrusion detection: once a node is compromised or a radio replays captured frames, the mesh silently loses data. We present MeshSentry, an intrusion detection system (IDS) that runs on the ESP32-S3 itself. Because every MeshSentry frame is an authenticated broadcast, each node passively monitors its neighbours. It applies a binomial hypothesis test to forwarding behaviour, an exponentially weighted baseline to frame rates, a sliding anti-replay window, and a check of hop-1 route claims against a sink-signed neighbour list. Nodes condense this evidence into 10-byte records. The sink fuses them by trust-weighted voting with an accuser quorum and detects Sybil identities by clustering their received-signal-strength (RSSI) signatures across monitors. In a 25-node simulation with five attacks (selective forwarding, sinkhole, flooding, replay, Sybil), MeshSentry detects 95–100% of attacks with a false-positive rate (FPR) of 0.4–3.7%, compared with 3.7–16% for the same detectors without fusion and 16–25% for a classical watchdog. Median detection latency is 16–26 s. Isolating detected nodes restores the packet delivery ratio from 67% to 90% under a sinkhole attack, and IDS reports occupy 0.8% of channel airtime. The complete firmware builds with ESP-IDF v6.1, and the IDS adds only 2.6 kB of flash and 2.7 kB of RAM to a sensor node.**

***Index Terms*—Intrusion detection, ESP-NOW, ESP32-S3, wireless mesh networks, selective forwarding, Sybil attack, trust management, IoT security.**

I. Introduction

Espressif's ESP-NOW protocol [1] exchanges short vendor-specific action frames between ESP32 devices without an access point or an association handshake. Its low latency, its range, and the low price of chips such as the ESP32-S3 [2] have made it a popular basis for home-automation, agricultural and industrial sensor meshes. These meshes are typically built from ad-hoc relaying code. The link-layer encryption that ESP-NOW optionally provides protects confidentiality between paired peers, but it does nothing against an *insider*: a node whose key has been extracted can drop the traffic it should relay, advertise forged routes, inject traffic, or claim several identities. Outsiders can still replay captured frames.

Intrusion detection for wireless sensor networks has a long history, including watchdogs [3], reputation systems [4], [5], 6LoWPAN-oriented IDSs such as SVELTE [6], and the surveys in [7], [8]. Most of this work assumes TinyOS-class motes or IP-based stacks with routers that can host detection logic. ESP-NOW meshes differ in three ways that we exploit: (i) frames can be broadcast so every neighbour overhears every hop; (ii) the ESP32-S3 has hardware SHA and big-number accelerators, so every frame can be authenticated and control messages can carry public-key signatures; and (iii) it has enough RAM to keep per-neighbour statistics for dozens of neighbours.

This paper makes the following contributions:

- **Design.** MeshSentry, a two-tier IDS for ESP-NOW meshes. Every node runs lightweight statistical detectors on overheard traffic, and a sink fuses compact evidence with trust-weighted, quorum-based voting and RSSI-signature Sybil detection (Sec. IV).
- **Principled local tests.** We replace the fixed forwarding-ratio threshold of the classical watchdog with a one-sided binomial test that is normalised by the measured link quality. At the same detection rate, this cuts the false-positive rate from 20% to 0.2% (Sec. VI).
- **Implementation.** An open ESP-IDF v6.1 firmware for the ESP32-S3 with PSA-Crypto frame authentication, ECDSA-signed control floods, and a testbed attack-emulation module (Sec. V).
- **Evaluation.** A simulation study of five attacks plus colluding bad-mouthing monitors, including the honest finding that reputation weighting does not help against colluders who build trust before they strike; the accuser quorum does (Sec. VI).

II. Related Work

Watchdogs and reputation. Marti et al. [3] proposed the watchdog/pathrater pair, in which a sender overhears its next hop and flags it when the fraction of forwarded packets falls below a fixed threshold. CONFIDANT [4] and RFSN [5] propagate Bayesian reputation, the latter using the Beta reputation system [9], which we also adopt for monitor trust. Fixed thresholds confuse link losses and collisions with misbehaviour. As our results show, this gives a watchdog false-positive rates near 20% in realistic channels.

IoT intrusion detection. SVELTE [6] places detection in the 6LoWPAN border router and targets RPL attacks (sinkhole, selective forwarding). Kalis [10] adapts its detection modules

to the network context it discovers. Surveys [7], [8] classify placement (centralised, distributed, hybrid) and methods (signature, anomaly, specification). MeshSentry is a hybrid, specification- and anomaly-based design that does not depend on IP routing.

Sybil and routing attacks. Karlof and Wagner [11] catalogue routing attacks on sensor networks, and Newsome et al. [12] analyse Sybil defences. Demirbas and Song [13] showed that ratios of RSSI values measured by several receivers identify Sybil identities that share one radio. We use a related RSSI-signature distance but compute it at the sink from evidence that monitors report anyway, so it adds no traffic. Signed control floods follow the authenticated-broadcast idea of SPINS [14], with ECDSA [15] instead of μTESLA, which the S3 can afford.

To the best of our knowledge, no prior work provides an IDS for ESP-NOW meshes, or an evaluated IDS implementation for the ESP32 family. [TODO: verify novelty claim against 2024–2026 literature]

III. System and Threat Model

Network. N static ESP32-S3 nodes report sensor data to a trusted sink over multiple hops. All frames are ESP-NOW broadcasts carrying the intended next hop, so any neighbour can overhear them. Routing is an ETX gradient [16]. Members share a network key K, which they use to authenticate frames with a truncated HMAC. The sink holds an ECDSA-P256 key pair whose public key is provisioned on all nodes.

Adversary. We consider (a) *insiders*: compromised members that know K and run arbitrary firmware; and (b) *outsiders* without K that can record and retransmit frames. The attacks are *selective forwarding* (dropping relayed frames with probability p), *sinkhole* (claiming a one-hop route to the sink and then dropping everything), *flooding* (authenticated junk at hundreds of frames/s), *replay* (verbatim retransmission of captured frames), and *Sybil* (beacons under several fabricated identities). We also consider *colluding bad-mouthers*: compromised monitors that report false evidence against one honest node. Physical-layer jamming and compromise of the sink are out of scope.

IV. MeshSentry Design

Fig. 1 gives an overview of the design. Time is divided into windows of $W = 10\,\text{s}$ aligned to the sink clock. Each signed control frame carries the sink's time, which nodes use to align their windows.

A. Authenticated Broadcast Mesh

Fig. 2 shows the frame format. A 16-byte header carries the transmitter `tx`, a per-transmitter sequence number `txseq`, the originator and its end-to-end sequence number, the next hop, the hop count, and the payload length. A 64-bit truncated HMAC-SHA256 tag covers the header and payload. Receivers check `txseq` against a 64-entry sliding window, as in IPsec anti-replay [17]. The upper bits of `txseq` hold a boot epoch stored in flash, so a rebooted node is not mistaken for a replayer. Beacons advertise the path ETX. A node that loses its route keeps beaconing with infinite cost (*route poisoning*), so its neighbours do not expect it to forward. Every 10 s the sink floods a control frame signed with ECDSA-P256. It contains the *authenticated sink-neighbour list* (ASNL), i.e. the nodes the sink actually hears, and the current blacklist.

B. Per-Neighbour Monitoring and Local Detectors

For each neighbour j, a monitor i keeps an exponentially weighted estimate of the beacon reception ratio $\hat{p}_j$ (the PRR of link $j \to i$). It also keeps the mean RSSI and frame rate in the current window and watchdog counters.

Selective forwarding. Whenever i sends or overhears a data or report frame whose next hop is j (and j is not the sink), it expects to overhear j forward that frame within 400 ms. Third-party monitors take part only if $\hat{p}_j \geq 0.7$. Over a sliding window of three windows, let h be the number of forwards expected and o the number actually overheard. If j is honest, each expected forward is overheard with probability at least $q\,\hat{p}_j$, where $q = 0.9$ conservatively accounts for losses on the upstream link. Monitor i raises an alert if

$$F_{\text{Bin}}\big(o;\, h,\, q\hat{p}_j\big) = \sum_{k=0}^{o} \binom{h}{k} (q\hat{p}_j)^k (1 - q\hat{p}_j)^{h-k} < \alpha, \qquad (1)$$

with $h \geq 6$ and $\alpha = 10^{-3}$. Unlike a fixed ratio threshold, (1) adapts to link quality and to the amount of evidence.

Flooding. The per-neighbour rate r_t (frames/s) is compared with an EWMA baseline μ and a mean absolute deviation σ:

$$r_t > \max(r_{\min},\ \mu_{t-1} + k \max(\sigma_{t-1}, 0.5))\,, \qquad (2)$$

with $k = 4$ and $r_{\min} = 5$ frames/s. While an alert is active, the baseline is not updated, so an attacker cannot drag the baseline upward.

Forged routes. A beacon that claims hop count 1 from a node absent from the last two signed ASNLs counts as a bad claim. An alert is raised after two bad claims in one window.

Replay. Frames that pass authentication but fail the freshness window are dropped and counted. Two or more in one window raise a replay alert. The alert names the identity being replayed but never blames it.

Local reaction. A node immediately stops routing through a neighbour for which it raised a forwarding alert, without waiting for the sink.

C. Evidence Reporting

At the end of each window, a node sends one report to the sink containing up to 16 ten-byte records (Fig. 2), with records that carry alerts first. The sink re-applies the forwarding test to the raw $(h, o, \hat{p})$ values, so thresholds can be retuned centrally without reflashing the nodes.

D. Sink Fusion

For a suspect j and alert class c, let E be the monitors whose records are *eligible* for c (e.g. $h \geq 6$ for forwarding),

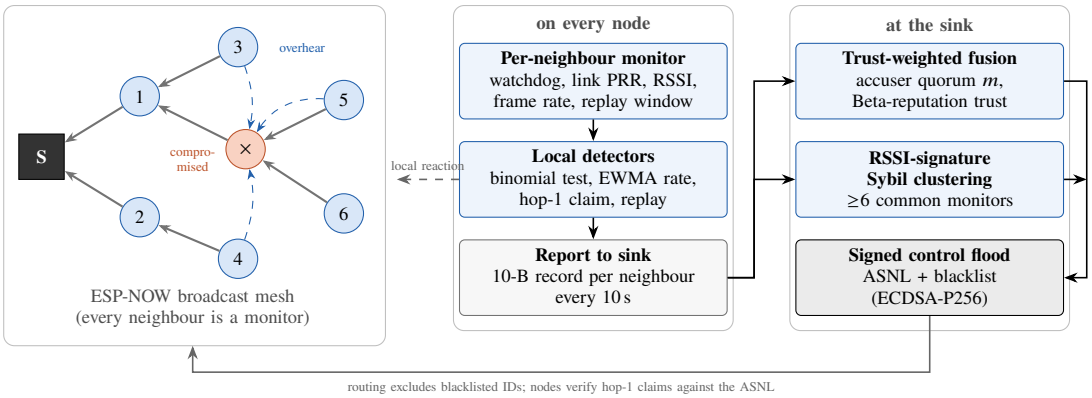


Fig. 1. MeshSentry architecture. Every node monitors the neighbours it overhears and runs local detectors. Every 10 s it sends one compact record per neighbour to the sink, which fuses the records with trust-weighted, quorum-based voting, clusters RSSI signatures to expose Sybil identities, and floods an ECDSA-signed control message carrying the authenticated sink-neighbour list (ASNL) and the blacklist.

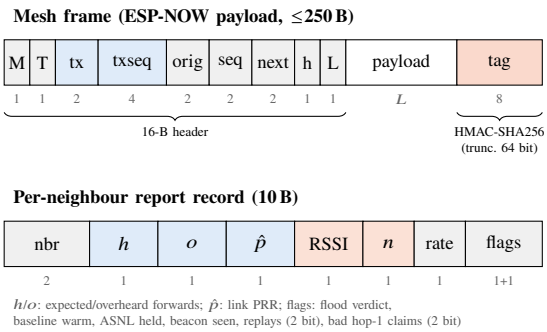


Fig. 2. MeshSentry frame and per-neighbour report record.

and let $A \subseteq E$ be those that accuse j. With monitor trust τ_i, the sink declares j malicious if

$$\frac{\sum_{i\in A}\tau_i}{\sum_{i\in E}\tau_i} \;\geq\; \theta_v, \qquad \sum_{i\in A}\tau_i \geq M, \qquad |A| \;\geq\; m, \tag{3}$$

with $\theta_v = 0.5$, $M = 0.9$, and an *accuser quorum* $m = 3$. For replay, the evidence is cryptographic, so the fraction condition is dropped. Trust is the expected value of a Beta reputation [9], $\tau_i = (a_i + 1)/(a_i + b_i + 2)$. A monitor's a_i (b_i) is incremented when it agrees (disagrees) with a clear-cut outcome, defined as a score ≥ 0.75 or ≤ 0.25.

Sybil detection. Identities that share one radio produce nearly identical mean RSSI at every monitor. For identities a, b heard by a common monitor set $\mathcal{M}_{ab}$ with $|\mathcal{M}_{ab}| \geq 6$, the sink computes

$$d_{ab} = \sqrt{\frac{1}{|\mathcal{M}_{ab}|}\sum_{i\in\mathcal{M}_{ab}} (\bar{r}_{ia} - \bar{r}_{ib})^2} \tag{4}$$

and flags both identities if $d_{ab} < \delta = 1.5\,\text{dB}$ in two of the last three windows. Monitors with $\tau_i < 0.4$ are excluded.

Response. Nodes that are declared malicious are added to the blacklist in the next signed control flood. Nodes then exclude them from parent selection.

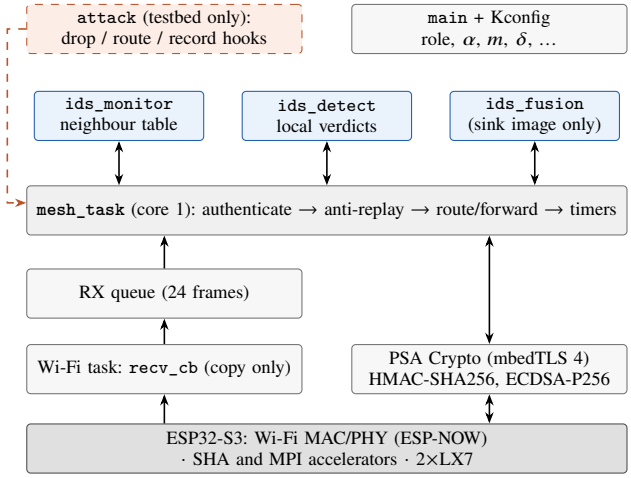


Fig. 3. Firmware architecture. The Wi-Fi task only copies received frames; all protocol and IDS state is owned by one task, so no locks are needed on the hot path.

V. Implementation on the ESP32-S3

The firmware (Fig. 3) targets ESP-IDF v6.1 and consists of five components: `proto` (wire format), `mesh` (ESP-NOW transport, routing, control plane), `ids` (monitor, detectors, fusion), `attack` (testbed emulation), and `main`. Authentication uses the PSA Crypto API of mbedTLS 4: `psa_mac_verify` with a truncated HMAC-SHA256 algorithm, which runs on the S3's SHA accelerator, and `psa_sign/verify_message` with ECDSA-P256 for control frames. The node role (sink, node, or attacker), the attack type, and every threshold (α, m, δ, …) are Kconfig options, and one script builds all seven role images. Fusion code and tables are compiled only into the sink image, so the IDS adds 2.6 kB of flash and 2.7 kB of RAM to a sensor node. The sink's fusion tables (reports from 32 monitors over two windows, trust, vote and Sybil-pair state) take 19 kB, which is under 6% of the S3's 512 kB SRAM.

Table I lists the memory footprint measured with `idf.py size-components`. Per-node IDS state is statically allocated:

TABLE I
FIRMWARE FOOTPRINT ON THE ESP32-S3 (BYTES, ESP-IDF V6.1, -O2); WHOLE-IMAGE RAM INCLUDES CODE PLACED IN INTERNAL RAM

Component	Flash code	Flash data	Static RAM
`ids` (sensor node)	2 688	0	2 762
`ids` (sink, with fusion)	5 553	0	19 106
`mesh` (sensor node)	4 934	114	1 973
Whole image (sensor node)	538 996	103 360	119 186

TABLE II
SIMULATION PARAMETERS

Topology	25 nodes, 5×5 grid, 15 m pitch, ±3 m jitter
Radio	0 dBm, PL_0=40 dB, n=3.3, σ_s=4 dB, σ_f=1.5 dB
Reception	sens. −88 dBm, CS −92 dBm, 1 Mb/s ESP-NOW
Traffic	beacon 2 s, data 5 s/node, report 10 s/node
Run	400 s, attack at 150 s, 20 seeds/scenario
Attacks	drop p=0.6; flood 300 f/s; replay 4 f/s; 4 Sybil IDs
IDS	W=10 s, α=10^{-3}, m=3, θ_v=0.5, δ=1.5 dB

24 neighbour entries, 48 pending watchdog expectations, and an anti-replay bitmap per neighbour. The attack module reproduces the simulated attackers with behaviour hooks (drop, route rewrite, raw-frame recording), so the same experiments can be run on hardware.

VI. EVALUATION

A. Methodology

We evaluate MeshSentry with a discrete-event simulator that models the same protocol and runs a Python reference implementation of the detectors that mirrors the C firmware's logic and thresholds. Table II lists the parameters. The radio model uses log-distance path loss with symmetric static shadowing [18], per-frame fading, and a sigmoid reception curve that reproduces the transitional region [19]. The MAC model includes carrier sensing, collisions with a 4 dB capture threshold, and half-duplex radios. The attacker (a single node) is compromised at t = 150 s. For selective forwarding and sinkhole it is the relay with the largest subtree, which is the worst case. Fig. 4 shows a sample topology.

All thresholds were calibrated on a separate set of ten seeds. Every result uses 20 fresh seeds per scenario. We report the *detection rate* (the fraction of runs in which the attacker is declared after onset) and the *false-positive rate* (FPR, the fraction of honest nodes that are ever declared malicious during the 400 s run). We compare against (i) *Local-only*: the same detectors, but any single honest monitor's alert counts; and (ii) the classical *Watchdog* [3]: a sender flags its next hop if fewer than 50% of the frames it handed over are overheard being forwarded.

B. Detection Performance

Fig. 5 summarises the main results. MeshSentry detects the attacker in 100% of runs for selective forwarding, flooding, replay, and Sybil, and in 95% of runs for sinkhole. Its FPR stays between 0.4% and 1.5%, except under flooding (3.7%).

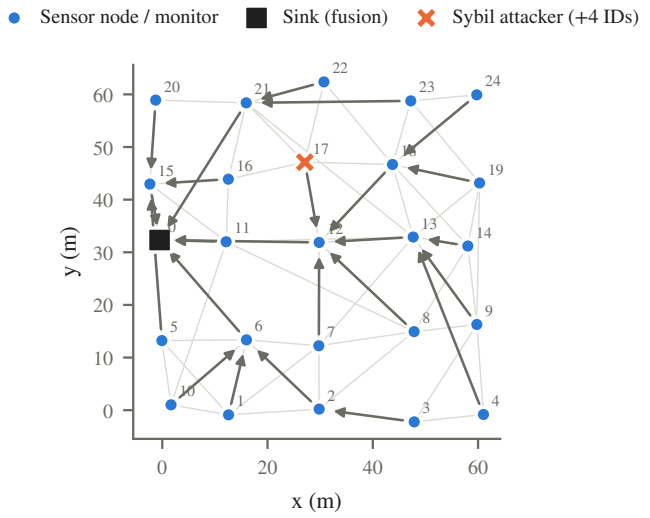


Fig. 4. A simulated topology. Arrows show the ETX routing tree towards the sink (node 0) and grey lines show usable links. In this seed node 17 is a Sybil attacker.

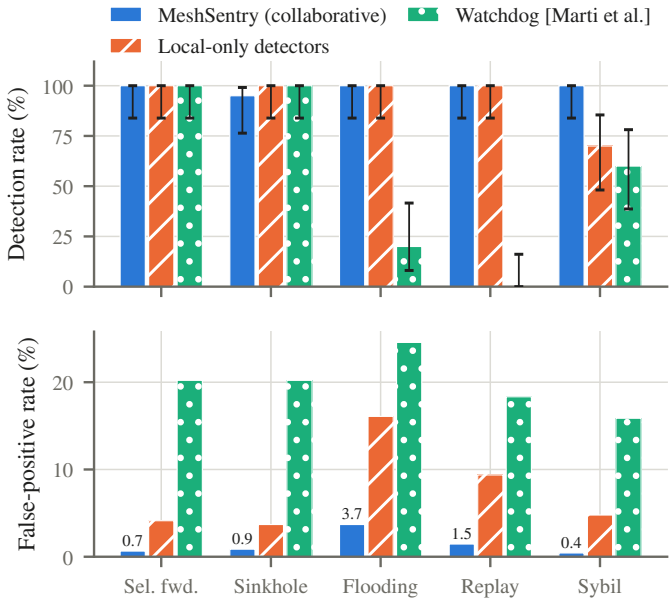


Fig. 5. Detection rate (top, 95% Wilson intervals) and false-positive rate (bottom) per attack, 20 runs each.

Without flooding, honest nodes are only rarely accused. Flooding raises the FPR because the collisions it causes make honest relays appear to drop frames, and that effect is real rather than a modelling artefact. Local-only detection has the same sensitivity but a 3.7–16% FPR: across all 120 runs, single-monitor forwarding alerts accuse 194 honest (run, node) pairs, versus 18 after fusion. Moreover, a single-monitor RSSI comparison cannot detect Sybil identities without flagging nearly every node. The watchdog misses flooding and replay entirely (it observes only forwarding), and its fixed threshold yields a 16–25% FPR.

Fig. 6 sweeps the forwarding test's significance level α

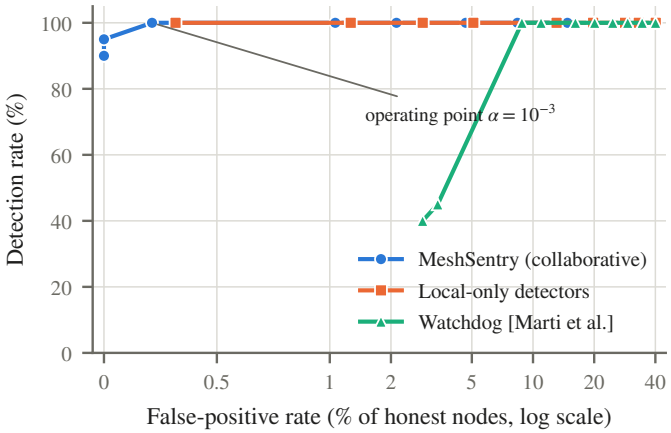


Fig. 6. Detection rate versus FPR for selective forwarding, obtained by sweeping $\alpha \in [10^{-6}, 0.4]$ (MeshSentry, local) and the ratio threshold (watchdog).

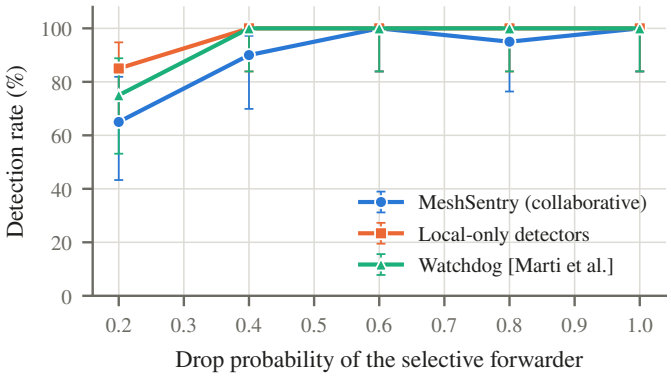


Fig. 7. Detection rate versus the selective forwarder's drop probability.

(and the watchdog's threshold) for selective forwarding. At $\alpha = 10^{-3}$, MeshSentry reaches 100% detection at 0.2% FPR. Reaching 100% detection with the watchdog costs 8.8% FPR. For forwarding alone, the binomial test is the main source of the gain: with a very strict α, local-only detection is equally good. The quorum $m = 3$ costs some sensitivity at very small α (90–95% detection), in exchange for resistance to bad-mouthing (Sec. VI-C) and a single network-wide verdict.

Fig. 7 varies the attacker's drop probability. Grayholes that drop 40% or more of relayed frames are detected in at least 90% of runs. A 20% grayhole is the hardest case: MeshSentry detects it in 65% of runs, compared with 85% for local-only detection. This is the expected cost of the quorum. Local-only detection, however, pays a 3–5% FPR, and the watchdog 13–14%, whereas MeshSentry stays below 0.7%.

Latency. Detection latency (Fig. 8) is quantised by the 10 s window plus the 6 s fusion delay. Flooding and replay are declared in the first fusion round (16 s). Forwarding attacks need evidence to accumulate ($h \geq 6$): their median latency is 26 s and the 90th percentile is 42 s.

Sybil signatures. Fig. 9 shows the distribution of d_{ab}. Of the pairs formed by Sybil identities and their host, 99.1% fall

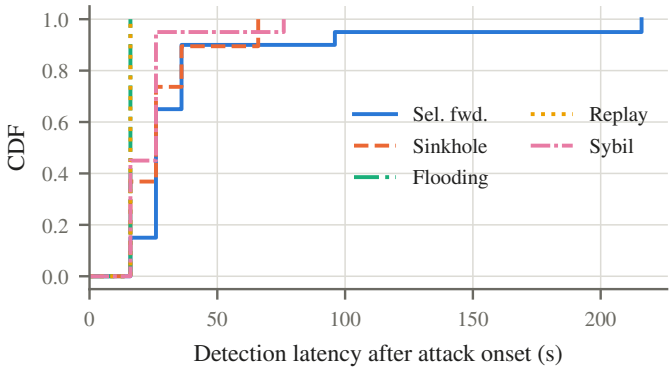


Fig. 8. CDF of detection latency after attack onset.

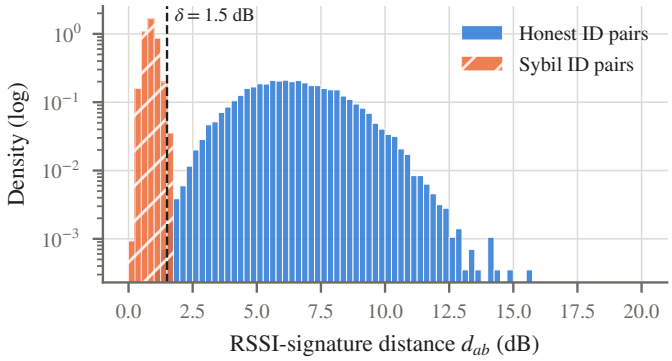


Fig. 9. RSSI-signature distance between identity pairs observed by at least six common monitors.

below $\delta = 1.5$ dB (99th percentile 1.46 dB), compared with only 0.03% of honest pairs. The two-of-three-windows persistence rule then removes the remaining honest coincidences.

C. Resistance to Bad-Mouthing

In the bad-mouthing experiment, a fraction of nodes collude. Each colluder reports fabricated evidence that one common honest victim is dropping frames, inventing records if necessary, while a selective forwarder is also active. Fig. 10 shows that the quorum $m = 3$ is the effective safeguard. With 10% colluders (two nodes), the FPR drops to 0.2%, compared with 4.7% for $m = 1$. Beyond 20% colluders, they outnumber the honest monitors eligible to judge the victim, and the FPR rises to 6–7%. Trust weighting gives *no* measurable benefit (the curves coincide). The colluders behave honestly until they strike, so they earn the same reputation as honest monitors, and a Beta reputation only discounts them once they are outvoted. We report this negative result because reputation is widely assumed to counter collusion.

D. Mitigation

With blacklisting and local reaction enabled, Fig. 11 shows the mean packet delivery ratio (PDR) of honest originators. Under the sinkhole attack, the PDR without isolation falls from 90.3% to 67.0%. With isolation it recovers within about 30 s to 89.8%, the pre-attack level. A 60% grayhole reduces the PDR to 78.9% without isolation and to 89.7% with it.

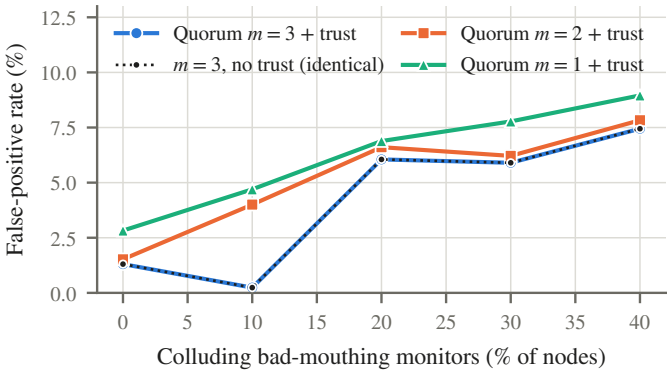


Fig. 10. FPR under colluding bad-mouthing monitors for different accuser quorums m.

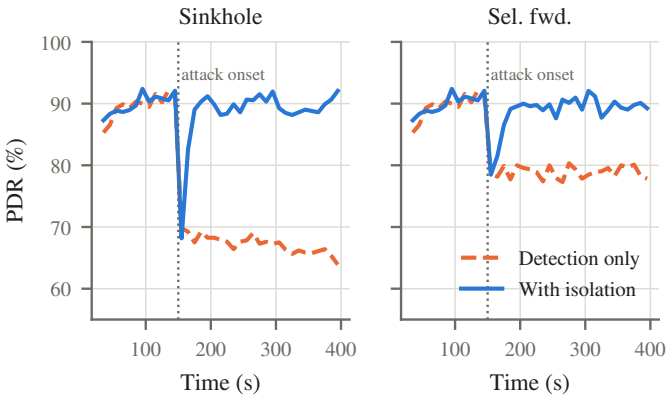


Fig. 11. Packet delivery ratio over time (mean of 20 runs) with and without isolation of detected nodes.

E. Overhead

Fig. 12 breaks the channel airtime down by frame type for networks of 16–49 nodes. IDS reports use 0.8% of airtime at 25 nodes and 2.0% at 49 nodes, which is comparable to the beacons. Total channel use stays below 7%. Reports grow faster than other traffic because they are relayed over more hops in larger networks. Sending reports every second window, or only when the content changes, would halve this cost. On the nodes, the IDS adds the memory listed in Table I and one HMAC verification per received frame, which is already required for authentication. [TODO: measure per-frame CPU time and current draw on hardware]

VII. Discussion and Limitations

Simulation versus hardware. All quantitative results come from simulation. The firmware is complete and compiles for every role, but has not yet been evaluated on a physical testbed. Real ESP-NOW links may show asymmetry, interference from Wi-Fi traffic, and RSSI quantisation that the channel model does not capture. [TODO: add 15–20 node ESP32-S3 testbed results using the provided attack images]

RSSI manipulation. A Sybil attacker that randomises its transmit power per identity can defeat absolute RSSI signatures. Using ratios of RSSI across monitors [13], or channel

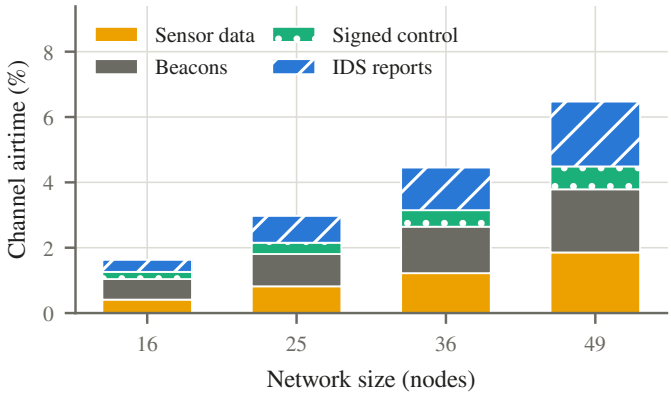


Fig. 12. Channel airtime by frame type versus network size (no attack).

state information (available on the ESP32-S3), is a natural extension. The Sybil detector also assumes static nodes.

Collusion. The accuser quorum bounds the damage of small coalitions, but colluders who outnumber a victim's honest neighbours can still frame it. Cross-checking accusations against the victim's own reports (for example, the symmetry of the RSSI the two endpoints report for their link) is future work.

Trust anchors. The sink and the provisioning of the network key are trusted. In deployment, the key would live in an eFuse key block used by the S3's HMAC peripheral rather than in flash.

VIII. Conclusion

MeshSentry shows that ESP-NOW meshes can detect insider and outsider attacks on the nodes themselves. Authenticated broadcast turns every neighbour into a monitor. A binomial forwarding test, a rate baseline, signed route references, and anti-replay windows provide cheap local evidence, and a quorum-based fusion at the sink turns that evidence into accurate, network-wide verdicts, including Sybil detection. In simulation, MeshSentry detects 95–100% of five attack types at 0.4–3.7% FPR, restores delivery after isolating attackers, and costs under 1% of airtime at 25 nodes. The next steps are a hardware testbed evaluation and RSSI-ratio Sybil signatures.

Artifact Availability

The ESP-IDF firmware, simulator, experiment scripts, and figure generation code are available at https://github.com/nlitayem/meshsentry-esp32s3.

References

[1] Espressif Systems, *ESP-NOW – ESP-IDF Programming Guide (v6.1)*, Espressif Systems, 2026.
[2] ——, *ESP32-S3 Series Datasheet*, Espressif Systems.
[3] S. Marti, T. J. Giuli, K. Lai, and M. Baker, "Mitigating routing misbehavior in mobile ad hoc networks," in *Proc. 6th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*, 2000, pp. 255–265.
[4] S. Buchegger and J.-Y. Le Boudec, "Performance analysis of the CONFIDANT protocol," in *Proc. 3rd ACM Int. Symp. Mobile Ad Hoc Networking and Computing (MobiHoc)*, 2002, pp. 226–236.

[5] S. Ganeriwal, L. K. Balzano, and M. B. Srivastava, "Reputation-based framework for high integrity sensor networks," *ACM Trans. Sensor Networks*, vol. 4, no. 3, 2008.

[6] S. Raza, L. Wallgren, and T. Voigt, "SVELTE: Real-time intrusion detection in the Internet of Things," *Ad Hoc Networks*, vol. 11, no. 8, pp. 2661–2674, 2013.

[7] I. Butun, S. D. Morgera, and R. Sankar, "A survey of intrusion detection systems in wireless sensor networks," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 266–282, 2014.

[8] B. B. Zarpelão, R. S. Miani, C. T. Kawakani, and S. C. de Alvarenga, "A survey of intrusion detection in Internet of Things," *J. Network and Computer Applications*, vol. 84, pp. 25–37, 2017.

[9] A. Jøsang and R. Ismail, "The beta reputation system," in *Proc. 15th Bled Electronic Commerce Conf.*, 2002.

[10] D. Midi, A. Rullo, A. Mudgerikar, and E. Bertino, "Kalis – a system for knowledge-driven adaptable intrusion detection for the Internet of Things," in *Proc. IEEE 37th Int. Conf. Distributed Computing Systems (ICDCS)*, 2017, pp. 656–666.

[11] C. Karlof and D. Wagner, "Secure routing in wireless sensor networks: Attacks and countermeasures," *Ad Hoc Networks*, vol. 1, no. 2–3, pp. 293–315, 2003.

[12] J. Newsome, E. Shi, D. Song, and A. Perrig, "The Sybil attack in sensor networks: Analysis & defenses," in *Proc. 3rd Int. Symp. Information Processing in Sensor Networks (IPSN)*, 2004, pp. 259–268.

[13] M. Demirbas and Y. Song, "An RSSI-based scheme for Sybil attack detection in wireless sensor networks," in *Proc. IEEE Int. Symp. World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2006, pp. 564–570.

[14] A. Perrig, R. Szewczyk, J. D. Tygar, V. Wen, and D. E. Culler, "SPINS: Security protocols for sensor networks," *Wireless Networks*, vol. 8, no. 5, pp. 521–534, 2002.

[15] National Institute of Standards and Technology, "Digital signature standard (DSS)," NIST, FIPS 186-5, 2023.

[16] D. S. J. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in *Proc. 9th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*, 2003, pp. 134–146.

[17] S. Kent, "IP encapsulating security payload (ESP)," RFC 4303, IETF, 2005.

[18] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2002.

[19] M. Zuniga and B. Krishnamachari, "Analyzing the transitional region in low power wireless links," in *Proc. 1st IEEE Int. Conf. Sensor and Ad Hoc Communications and Networks (SECON)*, 2004, pp. 517–526.